Week-3 Module-3

Q-1 Merge sort

→ Divide the array into smaller halves
→ sort each half recusively
→ Merges the sorted halves into one sorted array

Logic → Input the salary packages
call merge sort
Divide the array
Merge the Arrays

Dry Run ex → no of salary package = 5
Packages = 50, 20, 40, 10, 30

I-Array : [50, 20, 40, 10, 30]

Step 1: First function call
step 2: sort Left Half
Step 3: Merge (0, 0, 1)
step 4: Merge (0, 1, 2)
Step 5: sort Righ Half
step 6: final Merge (0, 2, 4)

Q-2

Algo

1- Read the no of order(n)
2- Read the processing times into an array
3- sort the array using Merge sort
4- Display the sorted processing time
5 -> Median
6 -> count how many processing time ax greater than the median
7-> Find diff b/w fates[1] and slow b✓

Time complexit = o(nlogn)

Q-3 Quick sort

Algo -> • Read the no of server (n)
• Read the server response times into an arr
• use quick to sort the arr
- choose the last element as pivot
- Partition the array so that elements smaller than t pivot are on t left t. —
4- Print the sorted list & ns1

Dry run →

step 1: quicksort (arr, 0, 5)

- Pivot = 60

Finally swap pivot with arr[i+1]:

Swap 120 & 60

Pivot index = 1

step 2: quicksort (arr, 2, 5)

(current array)

step 3: quicksort (arr, 4, 5)

Final sorted Arr

Q-4

logic → - Input

→ Calculate overall Average

→ Sort the Array in Descending order

→ Display sorted Trade val

→ find Top 5 Highest Trade val

→ count values above the overall

Dry run: Input

Overall Avg

Quick sort

Final sorted Arr

Top 5 Highest Trade val

count values above the overall

Q-5 Head sort

Step 1: Head sort
2: Build a Max Heap
3: Heap sort
4: Display Resut
5

Heapify Logic
↳ Assume the current node is the largest
→ Compare it with its left child
→ compare it with it right child